[redacted] – **Student ID:** [redacted]

1.)

Number of Leaves: 2^h

Base Case: When height is 0 (h = 0)

- $2^h = 2^0 = 1$
- This holds true as there is only one node, the root node for height = 0

Inductive Case: When height is > 0

- For height 1, the root would have two children (to make it a complete binary tree) and therefore there would be 2 leaves. This is also proven using the formula: $2^h = 2^1 = 2$
- *ASSUME:* 2^h is equal to the number of leaves for a complete binary tree of height h.
- *Prove* that for h + 1 the number of leaves would have to be 2^{h+1}
- If a tree of height h has 2^h leaves then when a new level is added there would have to be 2 nodes (the new leaves) added to each node of the previous level. This would mean that the new number of leaves would multiply by 2^1 which would make it such that $(2^1)2^h$ where the additional multiplication of base 2 would be added to the other, making 2^{h+1}. This proves that 2^h is the number of leaves for a complete binary tree. □

Number of Nodes: $2^{h+1} - 1$

Base Case: When height is 0 (h = 0):

- $2^{h+1} - 1 = 2^{0+1} - 1 = 2^1 - 1 = 2 - 1 = 1$
- This holds true as there is only one node, the root node for height = 0

Inductive Case: When height is > 0:

- For height 1, the root would have two children (to make it a complete binary tree) and therefore there would be 3 nodes. This is also proven using the formula: $2^{h+1} - 1 = 2^{1+1} - 1 = 2^2 - 1 = 4 - 1 = 3$
- *ASSUME:* $2^{h+1} - 1$ is equal to the number of nodes for a complete binary tree of height h.
- *Prove* that for h + 1 the number of nodes would have to be $2^{(h+1)+1} - 1$
- This formula would be simplified to $2^{h+2} - 1$
- If a tree of height h has $2^{h+1} - 1$ nodes, then if a new level were to be added below the last level, then to make it a complete binary tree, 2 nodes would have to be added below each node in the previous level, meaning that the number of nodes would multiply by 2^1 which would make it such that $(2^1)2^{h+1} - 1$ where the additional multiplication of base 2 would be added to the other, making $2^{h+2} - 1$. This proves that $2^{h+1} - 1$ is the number of nodes for a complete binary tree of height h. □

2.) Given that $L_n = L_{n-1} + L_{n-2}$, $n > 1$; $L_1 = 1$; $L_0 = 2$, prove that $L_n = F_{n-1} + F_{n+1}$, $n > 0$ where F_i is the Fibonacci number.

Base Case: Where n = 1

- $L_1 = F_{n-1} + F_{n+1} = F_{1-1} + F_{1+1} = F_0 + F_2 = 0 + 1 = 1$

Inductive Case: Where n > 1

- ASSUME: $L_n = F_{n-1} + F_{n+1}$ holds true
- Prove: if $L_n = F_{n-1} + F_{n+1}$ holds true then for one step higher, n + 1, $L_{n+1} = F_{n-1+1} + F_{n+1+1}$
- Simplified, this formula becomes $L_{n+1} = F_n + F_{n+2}$
- Because F_i is representative of the respective Fibonacci number, then based on the formula above: $F_n = F_{n+1} + F_{n-2}$ and $F_{n+2} = F_{n+1} + F_n$
- To prove that $L_{n+1} = F_n + F_{n+2}$ we use the initial Lucas number formula as well...
- $L_{n+1} = L_{n-1+1} + L_{n-2+1} = L_n + L_{n-1}$
- Based on the assumed formula, $L_n + L_{n-1} = (F_{n-1} + F_{n+1}) + (F_{n-2} + F_n) = (F_{n+1} + F_n) + (F_{n-1} + F_{n-2})$ which based on the formula for the Fibonacci numbers above is equal to $F_n + F_{n+2}$ and therefore $L_{n+1} = F_n + F_{n+2}$ □

3.) $\lg(n!) = \Theta(n\lg n)$

Upper Bound: of $\lg(n!) = \lg(n * n-1 * n-2 * ... * 1)$

- $\lg(n!) = \lg(n) + \lg(n-1) + \lg(n-2) + ... + \lg(1)$
- $\lg(n!) \leq \lg(n)_1 + \lg(n)_2 + \lg(n)_3 + ... + \lg(n)_n$
- This means that lg(n) will be performed nlg(n) times at most

Lower Bound: of $\log(n!) = \lg(n * n-1 * n-2 * ... * 1)$

- $\lg(n!) = \lg(n) + \lg(n-1) + \lg(n-2) + ... + \lg(1)$
- $\lg(n!) \geq \lg(n/2)_1 + \lg(n/2)_2 + \lg(n/2)_3 + ... + \lg(n/2)_{n/2}$ because lg(n!) would have to be bigger than half of its initial value.
- This means that lg(n) will be performed (n/2)lg(n) times at least

Because $(n/2)n\lg(n) \leq \lg(n!) \leq n\lg(n)$ then this means that lg(n!) is in between the bounds and $\lg(n!) = \Theta(n\lg n)$ □

4.) Exercise 2.3-4.

For each item in A, starting from the first element you sort into the previously sorted part of the list

$T(n) = \Theta(1)$ if n=1, $T(n-1) + \Theta(n)$ if n>1

- In a base case of only one element being in the list then then list would sort itself in 1 operation and therefore, $\Theta(1)$.

- If there are more than one element in the list, then after sorting the first element the algorithm would move to the next list element and would have to sort the rest of the list, meaning that it would have to reconduct the sort on n-1 elements of the list. (this becomes the recurrence T(n-1) + Θ(n))
- If every element to be sorted would be the smallest value of all the previously sorted items, so everything in the first portion of list A would be shifted to the right to accommodate the new smallest value. This would take Θ(n) operations.
- In the worst case, this step must be conducted for every list element, imply it must be done n times, meaning this would take $\Theta(n^2)$ operations.

Therefore in the worst case $\Theta(n^2)$ which would be expressed through T(n) = Θ(1) if n=1, T(n-1) + Θ(n) if n>1 □

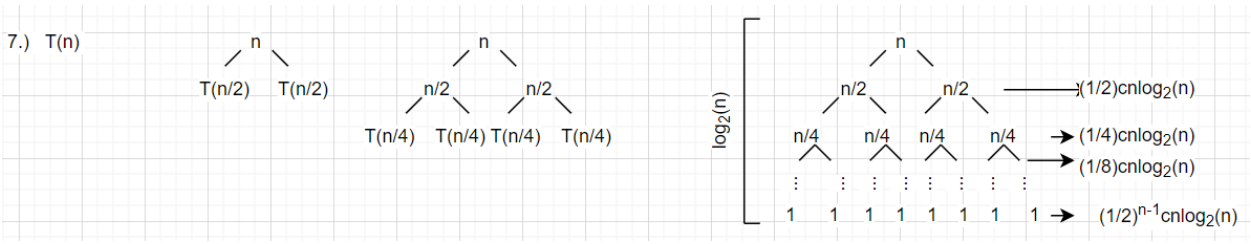
5.) Problems 3-2

A	B	*O*	*o*	*Ω*	*w*	*Θ*
$lg^k n$	$n^{\in}$	Yes	Yes	No	No	No
n^k	c^n	Yes	Yes	No	No	No
√n	$n^{sin(n)}$	No	No	No	No	No
2^n	$2^{n/2}$	No	No	Yes	Yes	No
$n^{lg(c)}$	$c^{lg(n)}$	Yes	No	Yes	No	Yes
lg(n!)	$lg(n^n)$	Yes	No	Yes	No	Yes

6.) Problems 4-5 b

- If all the chips are being pairwise tested, then at most the number of comparisons for n chips would be n/2.
- If you compare two chips and one or both is/are a good chip, then your comparisons will be completed within less than n/2 steps and you no longer need to check the rest of the chips.
- If you compare two chips and both are bad, then you get the next two and compare them, adding a comparison to your counter.
- If you have compared every chip expect for the last comparison, there are one of two options:
    - 1. If the number of chips was odd, then there will only be one chip left and that means that, that chip would have to be the good chip if no good chip was found prior. this would make it so that the number of comparisons would be (n/2)-1 which would be below the ⌊n/2⌋
    - 2. If the number of chips was even, then there would be two chips left to do the final comparison. Because, in this example, ⌊n/2⌋ is an even number then the floor would come out to be an integer and thus, all the comparisons would occur within n/2 (and therefore also ⌊n/2⌋) comparisons.
- This would effectively reduce the problem size to half or less.

□



for the problem above, wherever there is n by itself, it is representative of the $cnlog_2(n)$ portion of the recurrence. Therefore, the recurrence will be done in $log_2(n) * cnlog_2(n)$, ignoring constants, time which will leave $O(nlog_2^2(n))$.

Base case: When n = 2

- Then $nlog_2^2(n) = 2*log_2^2(2) = 2$
- This hold true based on the definition of T(n) where $T(2) \leq 2c$

Inductive case: When n > 2 or n < 2

- Then $nlog_2^2(n)$ should also hold for the original complexity $2T(n/2) + cn\ log_2(n)$
- $nlog_2^2(n) \leq 2T(n/2) + cn\ log_2(n)$
- $\leq 2(n/2)log_2^2(n/2) + cn\ log_2(n)$
- $= nlog_2^2(n/2) + cn\ log_2(n)$
- Because there is a very small gap between $nlog_2^2(n)$ and $nlog_2^2(n/2) + cn\ log_2(n)$ then T(n) = $O(nlog_2^2(n))$ and $O(nlog_2^2(n))$ should be very close to T(n)

□

8a.)

```
#include <stdio.h>
#include <stdlib.h>

long long int lucas(int n)
{
  if(n == 0)
  {
    return(2);
  }
  else if (n == 1)
  {
    return(1);
  }
  else
  {
    return(lucas(n - 1) + lucas(n - 2));
  }
}

int main(int argc, char *argv[])
```

```
{
   int x = atoi(argv[1]);
   printf("lucas(%d * 5) = %lld\n", x, lucas(x*5));
   return(0);
}
```

8b.) code included in file

8c.) The bash command revealed that the first recursive algorithm was much faster (occurs within a few milliseconds) in comparison to the second algorithm. The first algorithm has a time of around O(nlog(n)) which can be much faster than the second algorithm (which can take up to more than a few seconds, average was slightly higher than 1.4s) which has a time of O(n). The recursive nature of the first algorithm allows the calculation to be completed without having to go through every single value, while the second algorithm does not.

8d.) Given the program written in part a), but changing the data type used to an integer of 4 bytes, rather than a long long integer type of 8 bytes, this program would not be able to compute L_{50}. Because L_{50} would evaluate to a number that is over 2147483647 which is the largest integer than the int data type can hold. As a result, this value would not be able to be held by int. The program in 8b) would be able to hold the value of L_{50} because, even though it uses primitive types, it holds segments of the larger number in an array of the primitive type. With each 10 digits, the segment of the lucas number can be stored.